

Predictive KV Cache Amortization for Edge Language Model Inference on Mobile Devices

Abstract—On-device language models prepend environmental context prompts before generating responses, but this prefill phase scales as $O(N)$ with prompt length and dominates latency on mobile hardware. We present Context-Aware Predictive KV-Cache Priming (CAP-KVC), which uses environmental sensor signals—GPS, BLE proximity, and time of day—to predict the needed context and pre-compute key-value cache tensors during idle periods. At inference time, the cached state is restored in 2.74 ms, bypassing the context prefill entirely. On a OnePlus 11R with Qwen2.5-0.5B-Instruct (Q4_K_M), native-side `std::chrono` instrumentation confirms a $40.78\times$ prefill-specific speedup (343 ms vs. 14001 ms at $N \approx 500$ tokens) and a $4.24\times$ end-to-end reduction (4574 ms vs. 19408 ms). The system is implemented as an Android application with a C++/JNI inference backend based on `llama.cpp`, a bounded LRU cache manager supporting three simultaneous KV states within 118.7 MB native heap, and sensor-driven context routing with hysteresis-gated drift detection. A proof-of-concept multimodal intent layer (sEMG + gaze, late fusion) is evaluated on synthetic data and does not constitute clinical validation.

Index Terms—Key-value cache, edge inference, predictive caching, large language model, mobile systems, Android.

I. INTRODUCTION

Quantized sub-billion-parameter language models now run on mobile SoCs [4], [5], [2], but context-aware inference—where an environmental system prompt is prepended to the user’s input—introduces an $O(N)$ prefill cost that can dominate latency for prompts of several hundred tokens [22]. Applications requiring low-latency contextual responses, such as augmentative communication (AAC) devices [1], cannot tolerate this overhead.

Server-side KV cache reuse techniques—prefix sharing [9], paged attention [10], prompt caching [12]—exploit multi-tenant request streams unavailable on single-user edge devices, where each request typically reprocesses the full context.

We present **Context-Aware Predictive KV-Cache Priming (CAP-KVC)**, which decouples the prefill from the inference path on mobile hardware. CAP-KVC uses GPS, BLE proximity, and time-of-day signals to predict the needed semantic context, executes the prefill during idle periods, serializes the resulting KV cache to persistent storage, and restores it in under 3 ms at inference time. The system is implemented as a complete Android application with a C++/JNI backend based on `llama.cpp` [2], Kotlin orchestration for cache residency and sensor routing, and a proof-of-concept multimodal intent layer. The contributions are:

- 1) A predictive KV cache architecture achieving a $40.78\times$ prefill-specific speedup and $4.24\times$ end-to-end speedup on a OnePlus 11R (Qwen2.5-0.5B-Instruct Q4_K_M,

343 ms vs. 14001 ms prefill, 4574 ms vs. 19408 ms end-to-end), with 2.74 ms cache restoration.

- 2) An edge-native Android implementation with a scalar-only JNI boundary (9 native functions), reentrant-lock concurrency, and an LRU cache manager bounded to 3 KV states within 118.7 MB native heap.
- 3) A deterministic sensor-driven context routing engine with reliability-adaptive weighting and hysteresis-gated drift detection.

II. RELATED WORK

KV cache optimization. Token-level compression methods—Scissorhands [6], H₂O [7], SnapKV [14], StreamingLLM [8], KVQuant [18]—reduce within-sequence memory but operate within a single sequence. Server-side prefix reuse systems (SGLang RadixAttention [9], PagedAttention [10], Prompt Cache [15], CacheBlend [16]) and cross-session streaming (CacheGen [12], InfiniGen [11]) target cloud serving with shared request streams. CAP-KVC differs by using environmental sensor signals for cache state selection on a single edge device, where no shared request stream is available.

Edge LLM deployment. Post-training quantization [4], [5], architecture-aware design [19], and inference frameworks [2], [20] enable on-device execution. PowerInfer-2 [17] extends to 47B parameters via heterogeneous NPU/CPU scheduling. Alizadeh et al. [21] use flash memory for models exceeding DRAM capacity. Despite these runtime improvements, the $O(N)$ prefill remains a latency bottleneck when contextual prompts span hundreds of tokens.

III. SYSTEM DESIGN

CAP-KVC amortizes contextual prompt ingestion by executing it as a background process prior to user interaction. Fig. 1 presents the system architecture.

A. Problem Formulation

Let $C = \{c_1, \dots, c_M\}$ be a set of pre-configured semantic contexts, each associated with a system prompt p_i of N_i tokens. In standard retrieval-augmented generation, p_i is concatenated with the user’s intent tokens at inference time, incurring an $O(N_i)$ prefill cost. CAP-KVC reformulates this as a scheduling problem: predict the most likely context $\hat{c}$ from environmental signals, execute the prefill of $p_{\hat{c}}$ during idle periods, and persist the resulting KV cache tensors. At inference time, the system restores the pre-computed cache and

CAP-KVC System Architecture

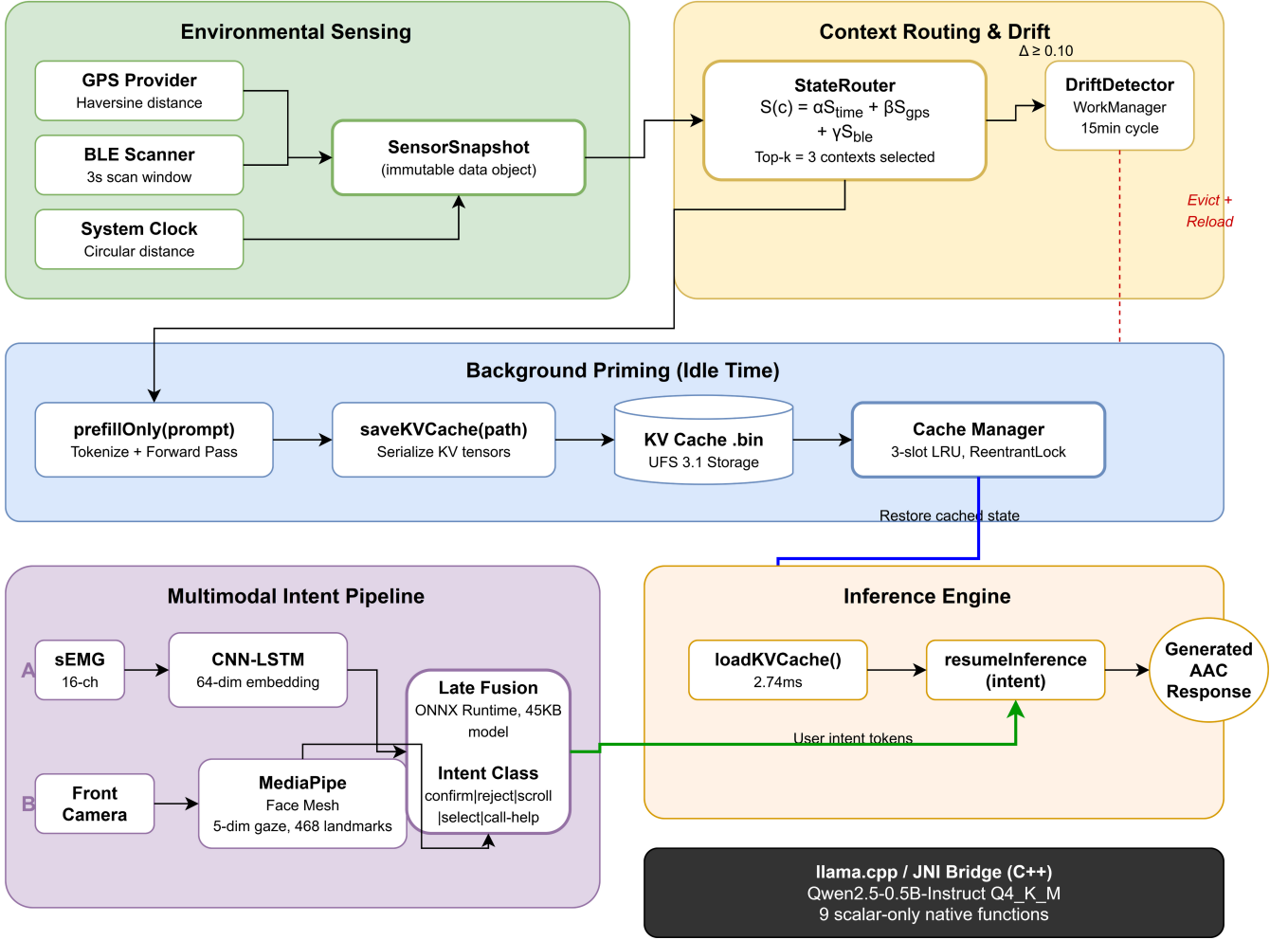


Fig. 1. CAP-KVC system architecture. Environmental sensors (blue) feed a scoring function selecting contexts for background priming (green). At inference time (orange), the pre-computed KV cache is restored and only intent tokens are processed by the llama.cpp/JNI backend. A drift detector re-evaluates context relevance with hysteresis gating ($\Delta \geq 0.10$).

appends only the user’s intent tokens, reducing the inference-time cost from a prompt-length-dependent prefill to a cache restoration operation whose cost is independent of the original prompt length.

B. Execution Pipeline

The system operates in five stages. The first three run asynchronously during idle periods: (1) *sensor acquisition* polls GPS, BLE, and the system clock to construct an immutable **SensorSnapshot**; (2) *context routing* evaluates all registered contexts via a convex scoring function (Section III-C) and selects the top- k for cache residency; (3) *background priming* executes the prefill for each selected context, serializes the KV tensors, and registers the context as resident. At interaction time: (4) *resident inference* restores the pre-computed cache and appends only the intent tokens; (5) *drift*

correction periodically re-evaluates scores and triggers cache replacement when a candidate exceeds the weakest resident state’s score by at least $\Delta = 0.10$, preventing cache thrashing under noisy sensor conditions.

C. Context Scoring

A context c_i is scored as a convex combination of environmental similarity:

$$S(c_i) = \alpha \cdot S_{\text{time}} + \beta \cdot S_{\text{gps}} + \gamma \cdot S_{\text{ble}} \quad (1)$$

where $\alpha + \beta + \gamma = 1$. Temporal similarity applies a Gaussian kernel ($\sigma = 2.0$ h) over circular time distance:

$$S_{\text{time}} = \exp\left(-\frac{d_{\text{circ}}(t, t_i)^2}{2\sigma^2}\right) \quad (2)$$

Spatial similarity uses exponential decay over Haversine distance ($\lambda = 0.1$ km):

$$S_{\text{gps}} = \exp(-\lambda \cdot d_{\text{haversine}}) \quad (3)$$

BLE similarity evaluates device-topology overlap with sigmoid-mapped RSSI values. Weights are computed dynamically from sensor reliability: time maintains a fixed baseline $w_t = 0.4$, guaranteeing a positive normalization denominator even when both GPS and BLE are unavailable. GPS reliability decays exponentially with reported accuracy ($w_g = \exp(-a/50.0)$); BLE reliability derives from mean sigmoid-transformed RSSI centered at -70 dBm. Missing sensors receive zero weight and remaining weights are renormalized to preserve $\alpha + \beta + \gamma = 1$. Contexts scoring below 0.01 are excluded.

D. KV Cache Residency

The system maintains at most $k = 3$ KV cache states simultaneously resident in native memory, mapped to native sequence slots managed by the inference engine. When all slots are occupied, the LRU state is evicted. Eviction is protected by reference counting: a state under active inference holds a reference blocking eviction until inference completes. The eviction sequence marks the victim inactive, polls until its reference count drains to zero, and returns the native slot to the available pool.

IV. IMPLEMENTATION

The system targets arm64-v8a devices running Android 12+, with `llama.cpp` [2] as the inference backend and Qwen2.5-0.5B-Instruct [3] (Q4_K_M) as the evaluation model. Pre-compiled native libraries are bundled in the APK. All dependency injection is manual to minimize startup latency.

JNI boundary. Nine native functions are exposed via JNI in a single C++ source file (801 lines), organized into three groups: lifecycle (`initializeBackend`, `initializeModel`, `release`), KV cache operations (`saveKVCache`, `loadKVCache`, `clearKVCache`), and inference (`runInference`, `prefillOnly`, `resumeInference`). A strict design constraint governs this boundary: **all arguments and return values are JNI scalar types** (`jstring`, `jint`, `jboolean`)—no Java objects, arrays, or callbacks cross the boundary, eliminating GC root pinning. The critical distinction is between `runInference` (clears session, processes from position zero) and `resumeInference` (appends at position n_{past}), preventing position-ID collisions that would corrupt the attention computation.

Concurrency. The C++ backend maintains shared global state (a `llama_context` pointer and `session_tokens` vector) accessed by the background priming coroutine, cache manager, and foreground inference path. A single `ReentrantLock` (the “engine lock”) guards all JNI call sequences. The lock is held across both `loadKVCache` and `resumeInference` within a single acquisition, because

releasing between calls would allow another thread to mutate `session_tokens`. Per-state coroutine `Mutex` instances separately protect cache residency metadata without blocking unrelated cache operations.

Cache priming lifecycle. Each state undergoes three phases: (1) prefill via `prefillOnly` under the engine lock, which runs the forward pass without entering the generation loop; (2) serialization via `saveKVCache` to a temporary file while still holding the lock (sequence slot 0 is mandatory since `prefillOnly` decodes into slot 0 via `llama_batch_get_one`); (3) atomic rename to the final path after lock release, exploiting `ext4`’s atomic `rename()` to guarantee crash consistency. A model-readiness gate (`AtomicBoolean`) prevents priming before native context initialization.

Sensor integration. Background sensing uses two cooperating components: an *ActiveSweep* coroutine that concurrently acquires GPS coordinates (via cached `getLastKnownLocation`) and BLE topology (via a 3-second `BluetoothLeScanner` window), and a *DriftDetector* `CoroutineWorker` registered with `WorkManager` for 15-minute periodic execution using a BLE-blind snapshot. A `BroadcastReceiver` for `BOOT_COMPLETED` and `MY_PACKAGE_REPLACED` triggers cache population at boot via the `goAsync()` pattern.

Inference fallback. If no cached context is available at interaction time (cold miss), a two-tier fallback fires: Tier 1 returns a generic pre-configured response immediately; Tier 2 initiates an asynchronous RAG prefill so subsequent intents can use contextual inference once the prefill completes.

V. EVALUATION

A. Setup

Hardware. OnePlus 11R (Snapdragon 8+ Gen 1, 8 GB LPDDR5X, UFS 3.1), Android 14, airplane mode, 50% brightness, no other foreground applications.

Model. Qwen2.5-0.5B-Instruct Q4_K_M via `llama.cpp` GGUF (`n_ctx=2048`, `n_batch=512`, `n_threads=4`, greedy decoding, max 64 output tokens).

Conditions. Three pipelines are compared: (1) ZERO_CONTEXT (intent tokens only, establishing the hardware latency floor); (2) RAG_INLINE (full context + intent via `runInference`, $N \in \{50, 100, 200, 500\}$); (3) CAP_KVC (restored cache via `loadKVCache` + intent via `resumeInference`). Each condition runs 32 sequential trials (2 warmup, 30 measured). The warmup phase absorbs Kotlin/ART JIT compilation and initial thermal settling.

Important limitation. CAP_KVC caches were derived from ~ 50 -token prompts. Comparisons against longer RAG_INLINE contexts characterize inference-time amortization behavior rather than response quality under equivalent context richness.

B. Measurement Methodology

End-to-end wall-clock latency is captured via `System.nanoTime()` at the Kotlin layer.

To decompose latency into constituent phases, `std::chrono::high_resolution_clock` timestamps are inserted immediately before and after each `llama_decode()` call within both `runInference` and `resumeInference`. This isolates the prefill phase—the single `llama_decode()` call that processes all prompt tokens—from the autoregressive generation loop. On the Android NDK runtime, `high_resolution_clock` maps to the monotonic `CLOCK_MONOTONIC` source (vDSO-accelerated, $\approx 10\text{--}50$ ns overhead), preventing corruption from NTP synchronization. JNI boundaries and log serialization occur strictly outside the timed regions.

For the CAP_KVC condition, three sub-components are instrumented separately: `cache_load_ms` (Kotlin layer), `prefill_ms` (native layer, intent-only `llama_decode`), and `gen_ms` (native layer, generation loop). Both layers are captured under the same engine lock acquisition.

Results are emitted as structured CSV via `adb logcat`. The extraction script matches JNI timing entries to Kotlin bench entries by strict proximity and asserts alignment of trial indices and token counts; any mismatch aborts the parser. All 180 trials reported here were fully verified.

VI. RESULTS

A. Inference Latency

Table I reports latency with native-side prefill decomposition ($n = 30$ per condition, 180 total).

TABLE I
LATENCY DECOMPOSITION (MS). PREFILL ISOLATES THE `LLAMA_DECODE()` FORWARD PASS; GENERATION COVERS AUTOREGRESSIVE SAMPLING. BOTH MEASURED VIA `NATIVE STD::CHRONO`. ONEPLUS 11R, QWEN2.5-0.5B-INSTRUCT Q4_K_M, GREEDY SAMPLING, MAX 64 OUTPUT TOKENS, $n = 30$.

Condition	E2E (ms)	Prefill (ms)	Gen (ms)	E2E σ	Tokens
ZERO_CONTEXT	3823.78	274.79	3544.58	569.76	9
RAG ($N \approx 50$)	6547.13	2021.46	4508.53	8392.58	52
RAG ($N \approx 100$)	7900.92	3015.09	4877.95	3857.18	89
RAG ($N \approx 200$)	12122.30	6673.53	5439.96	2740.12	206
RAG ($N \approx 500$)	19408.89	14001.46	5390.76	2113.04	439
CAP_KVC (cache load)	2.74	—	—	1.35	—
CAP_KVC (inference)	4571.97	343.35	4226.74	373.65	9
CAP_KVC (total)	4574.70	—	—	373.49	—

Cache restoration is negligible. The mean cache load time is 2.74 ms ($\sigma = 1.35$ ms), representing less than 0.1% of total CAP_KVC latency. This measures the complete Kotlin-side `loadKVCache()` round trip including JNI marshalling, native file I/O, and KV tensor deserialization.

Prefill amortization. RAG_INLINE prefill scales linearly from 2021 ms ($N \approx 52$) to 14001 ms ($N \approx 439$), a $6.93\times$ increase. CAP_KVC prefill is 343.35 ms ($\sigma = 45.00$ ms) for 9 intent tokens regardless of cached context size, yielding a **40.78 $\times$ prefill speedup** at $N \approx 500$. End-to-end, CAP_KVC achieves 4574.70 ms versus 19408.89 ms, a **4.24 $\times$ speedup**. The generation phase ($\sim 3500\text{--}5400$ ms) masks the prefill improvement in the end-to-end ratio.

Statistical significance. Welch’s two-sample t -tests ($\alpha = 0.05$, two-tailed) on prefill-only latency confirm significance: CAP_KVC vs. RAG@500 ($t(29.0) = -40.42$, $p < 10^{-26}$,

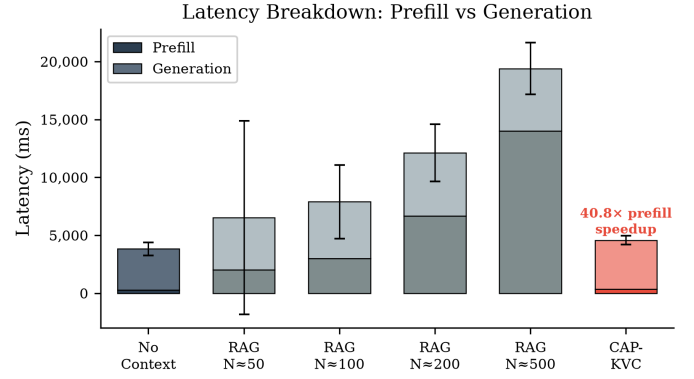


Fig. 2. Latency decomposition across conditions. RAG_INLINE prefill scales linearly (274 ms at $N=9$ to 14001 ms at $N=439$ tokens); CAP_KVC prefill is 343 ms regardless of cached context size, yielding a $40.78\times$ prefill speedup. Error bars: $\pm 1\sigma$, $n = 30$.

Cohen’s $d = 10.44$); CAP_KVC vs. RAG@50 ($p = 0.011$, $d = 0.70$, medium-to-large effect). The end-to-end comparison at $N \approx 500$ is also significant ($t(30.8) = -37.86$, $p < 10^{-26}$, $d = 9.78$); at $N \approx 50$ it is not ($p = 0.209$) due to high generation-phase variance. Although Welch’s t -test is robust to unequal variances, the extreme skewness of RAG latency distributions violates normality assumptions; the reported statistics should be interpreted as general trends.

Generation-phase variance. Generation time varies across conditions (ZERO_CONTEXT 3544 ms, RAG@500 5390 ms, CAP_KVC 4226 ms), consistent with differing per-token attention cost when the KV cache contains different numbers of entries and with uncharacterized system-level effects over the 29-minute benchmark session. Because prefill and generation are decomposed independently, these differences do not confound the prefill speedup.

Outliers. Five trials were flagged by Tukey’s criterion ($Q_3 + 3 \times \text{IQR}$). No thermal telemetry or CPU frequency logs were captured, so the specific mechanisms cannot be attributed. All outliers were retained in reported statistics to preserve operational realism.

B. Memory Budget

TABLE II
NATIVE HEAP MEMORY (KB) VIA `DUMPSYS MEMINFO`. DELTA REPRESENTS THE MARGINAL COST OF LOADING THREE KV CACHE STATES.

State	Native Heap (KB)	Total PSS (KB)
Idle (0 states)	18,856	623,488
Loaded (3 states)	140,420	612,940
Delta	+121,564 (≈ 118.7 MB)	—

Three-state residency adds approximately 118.7 MB of native heap (~ 40 MB per state for a 0.5B Q4_K_M model), a feasible budget on 8 GB devices. The slight PSS decrease between states reflects measurement variability in shared memory attribution.

C. Multimodal Fusion (Proof-of-Concept)

As a motivating application interface, a late fusion model combining CNN-LSTM sEMG embeddings ($\mathbb{R}^{64}$) with MediaPipe gaze features ($\mathbb{R}^5$) was evaluated on a synthetic dataset (2000 paired samples, 60/20/20 split). The pre-specified decision criterion required the cross-attention model to exceed late fusion F1 by more than 3 percentage points (absolute). Late fusion achieves macro F1 = 0.9963 versus cross-attention F1 = 0.9437 (5.26 pp difference, reversed direction), so the simpler architecture was selected. The standalone CNN-LSTM EMG classifier achieves 42.4% accuracy under cross-subject LOSO evaluation (NinaPro DB5 [13]), with 0% recall on the *reject* class—expected given inter-subject EMG variability. These numbers characterize model capacity on synthetic/controlled data and do not predict clinical accuracy; no gaze-only baseline was evaluated.

D. Drift Detection

An offline Sentence-BERT k -ablation on DailyDialog [23] yields maximum F1 = 0.367 at $k = 1$, degrading monotonically with window size. The deployed system uses $k = 3$ as a sensor-score smoothing heuristic—chosen for stability against transient sensor fluctuations rather than from the DailyDialog results, which are methodologically distinct (Sentence-BERT centroid distances vs. deployed sensor-score hysteresis).

VII. CONCLUSION

CAP-KVC demonstrates that sensor-driven predictive caching eliminates prompt-length-dependent prefill latency on mobile hardware. Native-side instrumentation confirms a $40.78\times$ prefill speedup (343 ms vs. 14001 ms at $N \approx 500$) and $4.24\times$ end-to-end reduction (4574 ms vs. 19408 ms), with 2.74 ms cache restoration. The Android implementation operates within 118.7 MB native heap for three simultaneous cache states. The approach generalizes beyond the AAC motivating scenario: any mobile application processing recurring environmental contexts can exploit idle-time cache priming.

Limitations. All benchmarks use a single device (One-Plus 11R, Snapdragon 8+ Gen 1) and a single model (Qwen2.5-0.5B Q4_K_M) with $n = 30$ trials; results may differ on other SoCs or model families. The fusion evaluation uses synthetic data and lacks clinical validation or a gaze-only baseline. Prefill timing measures the single `llama_decode()` call; tokenization overhead (< 1 ms) is excluded. No thermal telemetry, CPU frequency logs, power measurements, or non-predictive cache baseline were captured. The evaluation uses five static context configurations rather than learned profiles. Serialized KV cache files are unencrypted, relying on Android’s application sandbox. Battery impact of the ContextDaemon (60-second sweep) and DriftDetector (15-minute poll via WorkManager) has not been characterized.

ACKNOWLEDGMENT

The authors used Claude (Anthropic) to assist with drafting, editing, and language refinement of portions of this

manuscript. Google Antigravity was used as an AI-assisted software development tool during implementation of components of the Android/JNI system described in Section IV.

All research design, system architecture, implementation decisions, experimental methodology, benchmarking, statistical analysis, interpretation of results, and final manuscript verification were performed and validated by the authors. The authors accept full responsibility for the accuracy, integrity, and content of this work.

REFERENCES

- [1] D. R. Beukelman and J. C. Light, “Augmentative and alternative communication,” *Paul H. Brookes Publishing*, 5th ed., 2020.
- [2] G. Gerganov, “llama.cpp: LLM inference in C/C++,” 2023. [Online]. Available: <https://github.com/ggerganov/llama.cpp>
- [3] A. Yang *et al.*, “Qwen2.5 Technical Report,” *arXiv:2412.15115*, 2024.
- [4] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “LLM.int8(),” in *Proc. NeurIPS*, 2022.
- [5] E. Frantar, S. Ashkboos, T. Hoeffer, and D. Alistarh, “GPTQ,” in *Proc. ICLR*, 2023.
- [6] Z. Liu *et al.*, “Scissorhands,” in *Proc. NeurIPS*, 2023.
- [7] Z. Zhang *et al.*, “H₂O,” in *Proc. NeurIPS*, 2023.
- [8] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, “Efficient streaming language models with attention sinks,” *arXiv:2309.17453*, 2023.
- [9] L. Zheng *et al.*, “SGLang,” in *Proc. NeurIPS*, 2024.
- [10] W. Kwon *et al.*, “PagedAttention,” in *Proc. 29th SOSP*, 2023.
- [11] W. Lee, J. Lee, J. Seo, and J. Sim, “InfiniGen,” in *Proc. 18th USENIX OSDI*, 2024.
- [12] Y. Liu *et al.*, “CacheGen,” in *Proc. ACM SIGCOMM*, 2024.
- [13] M. Atzori *et al.*, “NinaPro,” *Scientific Data*, vol. 1, p. 140053, 2014.
- [14] Y. Li *et al.*, “SnapKV,” *arXiv:2404.14469*, 2024.
- [15] I. Gim *et al.*, “Prompt Cache,” in *Proc. MLSys*, 2024.
- [16] J. Yao *et al.*, “CacheBlend,” *arXiv:2405.16444*, 2024.
- [17] Z. Xue *et al.*, “PowerInfer-2,” *arXiv:2406.06282*, 2024.
- [18] C. Hooper *et al.*, “KVQuant,” in *Proc. NeurIPS*, 2024.
- [19] Z. Liu *et al.*, “MobileLLM,” in *Proc. ICML*, 2024.
- [20] MLC Team, “MLC LLM,” 2023. [Online]. Available: <https://github.com/mlc-ai/mlc-llm>
- [21] K. Alizadeh *et al.*, “LLM in a Flash,” *arXiv:2312.11514*, 2023.
- [22] A. Vaswani *et al.*, “Attention is all you need,” in *Proc. NeurIPS*, 2017.
- [23] Y. Li *et al.*, “DailyDialog,” *arXiv:1710.03957*, 2017.